

CS752 - Mini Project

a Report on

**DFARM: A DEADLINE AND FAULT-TOLERANT
AWARE RESOURCE MANAGER FOR EDGE
COMPUTING**

By

Ganji Sridhar Babu

Roll No.: 252CS012

M.Tech - 2nd Semester

Under the guidance of

Prof. Annappa B

Professor

Dept. of CSE, NITK

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**NATIONAL INSTITUTE OF TECHNOLOGY, KARNATAKA
SURATHKAL, MANGALORE - 575025**

May, 2026

Abstract

Edge computing enables real-time responsiveness for latency-sensitive applications but imposes severe constraints: limited virtual machine resources per node, wireless network delays, significant VM boot overhead, and distributed multi-node infrastructure. Existing cloud-optimized schedulers fail catastrophically at the edge due to assumptions of abundant resources, negligible network delays, and instant VM availability.

This project presents an edge-adapted implementation of DFARM (Deadline and Fault-Aware task Adjusting and Resource Managing), a task scheduling algorithm addressing edge computing challenges. The core innovation is network-aware completion time modeling that explicitly accounts for wireless transmission delays, inter-node transfer costs, VM cold-start overhead, and computational execution time factors systematically ignored by conventional cloud schedulers.

DFARM employs a four-phase scheduling strategy: direct assignment to active VMs, aggressive queue reordering, dynamic VM acquisition through a four-source cascade including cross-node offloading, selective replication for fault tolerance, and deadline-aware rejection. Tasks are prioritized using a deadline-per-length metric accounting for network transmission overhead.

The CloudSim-based implementation models a three-node edge topology with heterogeneous VMs serving burst workloads. Experimental evaluation against four baseline algorithms demonstrates that DFARM achieves 9-35% lower task rejection rates, 3-17 $\times$ faster makespan, 5-34 $\times$ higher throughput, and competitive resource utilization. Critically, DFARM is the only scheduler capable of cross-node offloading, successfully routing tasks to neighboring nodes when local resources saturate.

This work validates that explicit modeling of edge-specific constraints wireless latency, VM lifecycle overhead, distributed topology is essential for effective task scheduling in next-generation edge infrastructure.

Contents

1	INTRODUCTION	1
1.1	Background and Motivation	1
1.2	CloudSim Framework	2
1.3	Research Objectives	4
1.4	Report Organization	5
2	LITERATURE REVIEW	7
2.1	Cloud vs Edge Scheduling Paradigms	7
2.2	Baseline Algorithms	7
2.2.1	Minimum Completion Time (MCT)	7
2.2.2	RALBA	8
2.2.3	Round Robin and Random Selection	8
2.3	DFARM: Original Cloud Formulation	8
2.3.1	Core Algorithm Components	8
2.3.2	Experimental Configuration	10
2.3.3	Performance Results	11
2.3.4	Cloud-Specific Assumptions and Edge Adaptation Motivation	12
2.4	Research Gaps	12
3	SYSTEM DESIGN	14
3.1	Edge Topology	14
3.2	Network-Aware Completion Time Model	14
3.3	DPL-Based Prioritization	15
3.4	Five-Phase Scheduling Algorithm	15

3.4.1	Phase 1: Direct Assignment	15
3.4.2	Phase 2: Queue Adjustment	17
3.4.3	Phase 3: Dynamic VM Acquisition	18
3.4.4	Phase 4: Rejection (Hard-line mode)	20
3.5	Replication (15% ratio)	21
3.6	VM Lifecycle Management	22
4	IMPLEMENTATION	23
4.1	Class Architecture	23
4.2	Event-Driven Flow	24
4.3	Workload Design	25
4.3.1	Burst Arrival Pattern	25
4.3.2	Task Distribution	25
4.4	Baseline Implementations	26
4.5	Performance Metrics	26
5	RESULTS AND ANALYSIS	27
5.1	Experimental Parameters	27
5.2	Task Rejection Rate Analysis	29
5.3	Makespan Performance	30
5.4	Throughput Analysis	31
5.5	Resource Utilization	32
5.6	Cross-Node Offloading	33
5.7	Performance Summary	33
6	CONCLUSION AND FUTURE WORK	35
6.1	Contributions	35
6.2	Limitations/Future Work	36
6.3	Final Conclusion	36

Chapter 1

INTRODUCTION

1.1 Background and Motivation

Edge computing has emerged as a critical paradigm shift in distributed computing, driven by the explosive growth of Internet of Things devices, autonomous vehicles, augmented and virtual reality applications, and real-time analytics systems. Unlike traditional cloud computing architectures that centralize computational resources in large scale data centers, edge computing strategically positions computational infrastructure at the network edge typically at cell towers, base stations, and micro data centers to minimize end-to-end latency and reduce bandwidth consumption for latency sensitive applications [6].

This architectural shift introduces fundamental constraints that invalidate the design assumptions underlying cloud optimized task schedulers. Cloud data centers operate under a resource abundance paradigm where hundreds to thousands of homogeneous virtual machines with high speed internal networking (often exceeding 10 Gbps with sub-millisecond latency) can absorb workload fluctuations and handle burst arrivals through elastic scaling. In contrast, edge nodes function under severe resource scarcity, typically hosting only a handful of heterogeneous virtual machines with computational capacities varying by an order of magnitude. The network environment at the edge exhibits multi tier heterogeneity: end devices communicate with edge servers over bandwidth constrained wireless links subject to multi second transmission delays, while inter-node communication leverages higher capacity fiber backhaul that still introduces non negligible transfer overhead. Furthermore, the virtual machine lifecycle at the edge incurs substantial cold-

start penalties encompassing both boot time and acquisition delays that can dominate task execution time for short computational workloads.

Parameter	Cloud Data Center	Edge Node
VMs per location	50-500	4-6
VM homogeneity	High (uniform MIPS)	Low (100-1200 MIPS)
Internal network	10+ Gbps, μ ms latency	500 Mbps fiber (inter-node)
Device connectivity	N/A	10-50 Mbps wireless, 2-10s delay
VM cold-start	Negligible	3-15s (boot + acquisition)
Resource abundance	Virtually unlimited	Scarce, frequently exhausted

Table 1.1. Cloud vs Edge Infrastructure Comparison

These architectural differences expose critical deficiencies in conventional cloud schedulers when deployed at the edge. Algorithms such as Minimum Completion Time, Resource Aware Load Balancing Algorithm, Round Robin, and Random Selection were designed under cloud assumptions and consequently suffer catastrophic performance degradation in edge environments, achieving task rejection rates between 72 and 92 % under realistic edge workloads. Their failure stems from four fundamental oversights: first, they ignore wireless uplink and downlink transmission delays that can dominate total task completion time; second, they fail to model virtual machine boot and acquisition overhead that introduces multi-second startup penalties; third, they operate within single-node boundaries and cannot leverage cross-node offloading opportunities when local resources saturate; and fourth, they lack deadline-driven prioritization mechanisms essential for time sensitive edge applications where missing deadlines renders task completion meaningless.

1.2 CloudSim Framework

CloudSim version 3.0.3 serves as the simulation foundation for this research, providing a mature discrete-event simulation toolkit specifically designed for modeling cloud and edge computing environments without the overhead and complexity of deploying physical

infrastructure [3]. Developed at the Cloud Computing and Distributed Systems Laboratory at the University of Melbourne, CloudSim enables researchers to evaluate resource provisioning algorithms, scheduling policies, and system architectures under controlled, repeatable experimental conditions while supporting precise temporal modeling of system behavior.

The framework’s architecture revolves around five core abstractions that together model the complete computational stack. The Datacenter abstraction represents physical infrastructure housing computational resources, encapsulating properties such as virtual machine allocation policies, network bandwidth characteristics, and energy consumption models. Within each datacenter, the Host abstraction models individual physical servers characterized by processing elements representing CPU cores with defined computational capacity measured in MIPS (Million Instructions Per Second), along with memory, storage, and bandwidth resources. The VM abstraction encapsulates virtualized compute instances that execute on hosts, each configured with specific MIPS ratings, core counts, memory allocations, and storage capacity. Computational workloads are represented by the Cloudlet abstraction, which models individual tasks characterized by computational length measured in MI (Million Instructions), input and output data sizes, and in our edge-adapted extension, deadline constraints and arrival times. Finally, the Datacenter-Broker abstraction serves as the scheduling intermediary responsible for virtual machine creation, cloudlet submission, and implementing task to VM assignment logic this is the primary extension point where custom scheduling algorithms are implemented by subclassing the broker and overriding its scheduling methods.

CloudSim’s execution model operates on a discrete-event simulation engine where simulation time advances in discrete steps triggered by specific events such as VM_CREATE indicating virtual machine instantiation, CLOUDLET_SUBMIT representing task arrival and submission, and CLOUDLET_RETURN signaling task completion. This event-driven architecture enables accurate modeling of asynchronous system behaviors including staggered task arrivals, network transmission delays, and resource contention dynamics, all without incurring wall-clock execution time proportional to simulated duration. For this edge computing research, we extended CloudSim’s base classes to incorporate edge-specific parameters absent from the original cloud-focused framework: wireless bandwidth charac-

teristics for device-to-edge communication, inter-node fiber backhaul for cross-node transfers, virtual machine boot times and acquisition delays modeling cold-start overhead, and sophisticated VM lifecycle state management tracking transitions between available, active, and released-idle states to support VM reuse optimization.

1.3 Research Objectives

This project pursues four interconnected research objectives that collectively advance the state of task scheduling for edge computing environments. The primary objective centers on edge adaptation of the DFARM algorithm, which requires developing and integrating network-aware completion time prediction formulas that explicitly model wireless transmission latency between end devices and edge servers, inter-node transfer costs when tasks offload across the distributed topology, and virtual machine acquisition delays encompassing both boot time for VM initialization and acquisition overhead for provisioning fresh instances from resource pools. This network aware modeling represents a fundamental departure from cloud based completion time predictions that assume negligible communication overhead.

The second objective addresses cross-node offloading capability, implementing dynamic load balancing mechanisms that enable task migration across the distributed edge topology when local virtual machine resources reach exhaustion. This capability is absent from all conventional baseline schedulers that operate within single-node boundaries, yet it is essential for edge environments where individual nodes' limited resource capacity frequently saturates under burst workloads. Successfully achieving cross-node offloading requires careful modeling of inter-node network costs, coordination across distributed node resource pools, and intelligent decision logic that balances the overhead of remote execution against the cost of local task rejection.

The third objective establishes rigorous comparative evaluation methodology by benchmarking DFARM's edge-adapted implementation against four established baseline algorithms - Minimum Completion Time, Resource Aware Load Balancing Algorithm, Round Robin, and Random Selection across systematically varying workload intensities ranging from two hundred to eight hundred tasks. This comprehensive evaluation spans multi-

ple workload scales to characterize performance under light, moderate, and heavy load conditions, revealing how algorithms behave as resource pressure intensifies.

The fourth objective focuses on performance validation across multiple dimensions, demonstrating DFARM’s superiority through quantitative analysis of task rejection rate measuring deadline satisfaction, makespan capturing overall execution efficiency, throughput quantifying successful task processing rate, and average resource utilization ratio assessing load balance quality. Together, these metrics provide a holistic view of scheduler performance that captures both user-facing service quality and system-level resource efficiency.

1.4 Report Organization

The remainder of this report systematically develops the DFARM edge scheduler from theoretical foundations through empirical validation. Chapter 2 surveys the relevant literature on cloud and edge task scheduling, examining the fundamental differences between cloud and edge paradigms, analyzing the design and limitations of existing scheduling algorithms including Minimum Completion Time, Resource Aware Load Balancing Algorithm, Round Robin, and Random Selection, reviewing the original cloud-based DFARM formulation, and identifying critical research gaps that motivate this edge adaptation work. Chapter 3 presents the complete system design including the three-node edge topology architecture, the network-aware completion time model incorporating wireless and inter-node delays, the deadline-per-length prioritization metric, the five-phase scheduling algorithm encompassing direct assignment, queue adjustment, dynamic VM acquisition with cross-node offloading, replication, and rejection phases, and the virtual machine lifecycle management strategy enabling VM reuse optimization. Chapter 4 details the CloudSim-based implementation including the six custom Java classes extending CloudSim’s base framework, the event-driven scheduling flow integrating DFARM with CloudSim’s discrete-event engine, the realistic burst workload generation strategy creating stress-test scenarios, the baseline scheduler implementations ensuring fair comparison, and the comprehensive performance metrics definitions. Chapter 5 analyzes experimental results across all evaluated algorithms and workload scales, presenting comparative

analysis of task rejection rates, makespan efficiency, throughput sustainability, resource utilization patterns, and the unique cross-node offloading capability that differentiates DFARM from baseline approaches. Finally, Chapter 6 synthesizes the research contributions, acknowledges current limitations including deferred node failure simulation and unenforceable replica diversity.

Chapter 2

LITERATURE REVIEW

2.1 Cloud vs Edge Scheduling Paradigms

Cloud Schedulers prioritize throughput and resource efficiency under abundant resources. FCFS, SJF, and Priority Scheduling assume negligible network delays and instant VM availability [5].

Edge Constraints invalidate these assumptions:

- **Resource Scarcity:** 4-6 VMs/node vs 50-500 in cloud
- **Network Heterogeneity:** 10-50 Mbps wireless + 500 Mbps fiber backhaul
- **Deadline Sensitivity:** Sub-second to few-second response requirements
- **Geographic Distribution:** Multi-node coordination required

2.2 Baseline Algorithms

2.2.1 Minimum Completion Time (MCT)

Approach: Greedy assignment to VM with earliest CT [2].

Limitations: Ignores deadlines, overloads fast VMs, achieves 87% rejection at 800 tasks.

2.2.2 RALBA

Approach: Two-phase fill-and-spill strategy [4].

- Fill: Assign large tasks evenly based on $\text{VMShare} = \text{totalMI} / \text{totalMIPS}$
- Spill: Overflow tasks via MCT

Limitations: Deadline-agnostic, no cross-node capability, 92% rejection rate.

2.2.3 Round Robin and Random Selection

RR: Cyclic VM assignment. **RS:** Random VM selection.

Limitations: Both fail catastrophically under bursts (88% rejection, 2100s makespan at 800 tasks).

2.3 DFARM: Original Cloud Formulation

Awan et al. [1] introduced DFARM (Deadline and Fault-Aware Resource Manager) as a comprehensive scheduling framework designed to address the dual challenges of deadline satisfaction and fault tolerance in cloud computing environments. The algorithm represents a significant advancement over traditional cloud schedulers by explicitly incorporating temporal constraints into resource allocation decisions while simultaneously providing robustness against infrastructure failures through selective task replication.

2.3.1 Core Algorithm Components

The DFARM framework comprises four interconnected mechanisms that together achieve superior deadline satisfaction compared to conventional cloud schedulers. First, the algorithm employs Deadline Per Length (DPL) prioritization as its fundamental task ordering metric, computed as the ratio of available execution slack (deadline minus arrival time) to computational workload magnitude (measured in million instructions). Mathematically, this is expressed as $\text{DPL} = (\text{deadline} - \text{arrivalTime}) / \text{MI}$, where lower DPL values indicate tighter temporal urgency. This normalization by workload size ensures that both

large computationally intensive tasks and small latency-sensitive tasks receive appropriate prioritization based on their relative urgency rather than absolute deadline proximity. By sorting incoming tasks in ascending DPL order, DFARM ensures that the scheduler addresses the most time critical workloads first, maximizing the probability of meeting stringent deadlines under resource contention.

Second, DFARM implements an aggressive queue adjustment mechanism that attempts to accommodate new tasks even when direct assignment would violate deadlines. When a task cannot be immediately assigned to any virtual machine without deadline violation, the scheduler examines the execution queues of all VMs and attempts to insert the new task ahead of lower priority (higher DPL) queued tasks. This insertion is validated by verifying that both the newly inserted task and all subsequently displaced tasks in the reordered queue still meet their respective deadlines. This queue manipulation represents a departure from conventional first-come-first-served queue semantics and enables DFARM to recover tasks that would otherwise be rejected, particularly under moderate load conditions where VM queues contain a mix of urgent and less time critical workloads.

Third, the framework incorporates dynamic virtual machine acquisition to expand available computational capacity on-demand when existing active VMs cannot accommodate incoming tasks within deadline constraints. Unlike static provisioning approaches that maintain a fixed VM pool regardless of workload dynamics, DFARM maintains a reserve pool of unallocated VMs and activates them selectively when deadlines cannot be met through assignment to existing resources. This dynamic provisioning strategy balances resource efficiency (avoiding over-provisioning during light load) with responsiveness (preventing deadline violations during burst arrivals) by adapting the active VM count to current workload pressure.

Fourth, DFARM implements selective task replication to provide fault tolerance against virtual machine or physical host failures. The algorithm identifies tasks in the bottom thirty percent of the DPL distribution (the most deadline-critical workloads) and creates duplicate replicas that are scheduled independently through the same assignment, queue adjustment, and acquisition phases. When the primary task or any of its replicas completes successfully, all remaining replicas are cancelled, ensuring that at least one instance completes even if individual VMs fail during execution. This selective replica-

tion strategy concentrates fault-tolerance overhead on the most critical tasks rather than uniformly replicating all workloads, thereby minimizing resource waste while protecting deadline sensitive operations.

2.3.2 Experimental Configuration

The original DFARM study conducted extensive simulation-based evaluation using CloudSim to model a large scale cloud data center environment. The experimental infrastructure comprised five hundred heterogeneous virtual machines distributed across fifty physical hosts, each host equipped with dual-core processors ranging from 1000 to 3000 MIPS computational capacity. Virtual machines were configured with varying resource profiles spanning 500 to 2000 MIPS per VM core, 512 MB to 4 GB memory, and 1 GB to 100 GB storage, reflecting the heterogeneity characteristic of modern Infrastructure-as-a-Service cloud platforms.

The workload consisted of synthetically generated task sets ranging from 100 to 1000 cloudlets with computational lengths varying from 10,000 to 400,000 million instructions, representing diverse application profiles from lightweight web service requests to compute-intensive batch analytics jobs. Task deadlines were assigned using multiple strategies to test scheduler robustness: tight deadlines set at $1.2\times$ minimum possible completion time to stress deadline satisfaction, moderate deadlines at $2.0\times$ minimum completion time representing realistic service-level objectives, and relaxed deadlines at $5.0\times$ minimum completion time providing substantial execution slack. This multi-level deadline strategy enabled comprehensive evaluation across varying temporal constraint regimes.

The evaluation compared DFARM against five established baseline algorithms representing the state-of-practice in cloud task scheduling: First-Come-First-Served (FCFS) providing baseline fairness without optimization, Shortest Job First (SJF) prioritizing workload completion rate, Minimum Completion Time (MCT) greedily minimizing individual task completion times, Max-Min algorithm balancing load across VMs, and HEFT (Heterogeneous Earliest Finish Time) optimizing for heterogeneous resource environments. Each algorithm was evaluated across identical workload scenarios with consistent infrastructure configurations to ensure fair comparative analysis.

2.3.3 Performance Results

The experimental results demonstrated DFARM’s substantial superiority across multiple performance dimensions under realistic cloud workloads. For deadline satisfaction, DFARM achieved task acceptance rates (percentage of tasks meeting deadlines) ranging from 87% under tight deadline constraints to 98% under moderate deadlines and near-perfect 99.5% under relaxed deadlines. In contrast, the best-performing baseline algorithm, MCT, achieved only 68% acceptance under tight deadlines and 89% under moderate constraints, representing a 19-9 percentage point improvement for DFARM. The deadline-agnostic algorithms FCFS and SJF performed catastrophically, achieving below 50% acceptance rates under tight constraints due to their complete disregard for temporal requirements.

In terms of resource utilization efficiency, DFARM maintained average resource utilization ratios between 0.82 and 0.91 across varying load intensities, indicating effective load balancing without excessive VM over-provisioning. The dynamic acquisition mechanism contributed to this efficiency by activating additional VMs only when necessary to meet deadlines, avoiding the resource waste inherent in static over-provisioning strategies. Makespan measurements showed DFARM completing 1000-task workloads approximately 23% faster than MCT and 35-42% faster than FCFS and SJF, demonstrating that deadline-aware prioritization accelerates overall workflow completion by ensuring critical-path tasks execute promptly.

The fault tolerance evaluation subjected the system to random VM failure events occurring at rates of 5%, 10%, and 15% of total VMs during execution. Under 10% failure rates, DFARM with 30% selective replication maintained 94% task success rates (tasks completing despite failures), while baseline algorithms without replication experienced catastrophic success rate degradation to 71-78%. This demonstrated that selective replication of deadline-critical tasks provides robust fault tolerance while consuming fewer resources than uniform replication strategies.

2.3.4 Cloud-Specific Assumptions and Edge Adaptation Motivation

Critically, the original DFARM formulation operates under three fundamental assumptions valid in cloud environments but violated at the edge. First, the completion time model assumes negligible network delay, computing task completion time purely as $CT = MI/MIPS$ without accounting for data transmission overhead a reasonable approximation in cloud data centers with multi-gigabit internal networks and co-located storage, but grossly inaccurate at the edge where wireless device-to-server communication incurs multi-second delays. Second, the VM lifecycle model assumes instant availability, treating VM acquisition as instantaneous without modeling boot time or provisioning delay valid when activating pre-warmed VMs in large cloud pools, but invalid at the edge where limited VM capacity necessitates frequent cold starts with substantial initialization overhead. Third, the topology model assumes a single centralized data center with uniform resource access, ignoring geographic distribution and inter node coordination appropriate for cloud’s centralized architecture but fundamentally mismatched to edge computing’s inherently distributed multi-node infrastructure.

These discrepancies between cloud assumptions and edge reality motivate the comprehensive adaptation presented in this work, requiring fundamental redesign of the completion time model, VM lifecycle management, and resource acquisition cascade to accurately capture edge specific constraints and enable effective deadline-aware scheduling in resource constrained, network-heterogeneous, geographically distributed edge environments. **Gap:** Assumes negligible network delays, instant VM availability, single datacenter all invalid at edge.

2.4 Research Gaps

1. **No Network-Aware Models:** Schedulers ignore 2-10s wireless delays and inter-node transfer costs.
2. **No Cross-Node Offloading:** Single-node operation causes rejection instead of neighbor offloading.

3. **Ignored Cold-Start:** 3-8s boot + 8-15s acquisition unmodeled.
4. **Inefficient Replication:** Cloud's 30% ratio wastes scarce edge resources.

Chapter 3

SYSTEM DESIGN

3.1 Edge Topology

Node	VMs	MIPS Range	Boot/Acq Delay	Wireless BW
Node A	4	100-500	5s / 10s	50 Mbps
Node B	6	500-1200	3s / 8s	35 Mbps
Node C	4	100-750	8s / 15s	10 Mbps
Inter-node backhaul: 500 Mbps fiber				

Table 3.1. Three-node edge infrastructure configuration

3.2 Network-Aware Completion Time Model

Edge-Adapted CT Formula:

$$CT = t_{\text{uplink}} + t_{\text{boot}} + t_{\text{acq}} + \frac{MI}{MIPS} + t_{\text{downlink}} \quad (3.1)$$

Component Definitions:

- $t_{\text{uplink}} = \text{inputSize}/\text{wirelessBW}$ — Device to edge transmission
- t_{boot} — VM boot time (3-8s)
- t_{acq} — Acquisition delay (0s for reused, 8-15s for fresh VMs)

- $t_{\text{compute}} = \text{MI}/\text{MIPS}$ — CPU execution
- $t_{\text{downlink}} = \text{outputSize}/\text{wirelessBW}$ — Result delivery

Cross-Node Extension:

$$t_{\text{interNode}} = \text{inputSize}/\text{fiberBW} \quad (3.2)$$

Contrast with cloud DFARM: $\text{CT}_{\text{cloud}} = \text{MI}/\text{MIPS}$ (ignores all network overhead).

3.3 DPL-Based Prioritization

Network-Aware DPL:

$$\text{DPL} = \frac{\text{deadline} - \text{currentTime} - t_{\text{uplink}}}{\text{MI}} \quad (3.3)$$

Lower DPL = tighter urgency. Tasks sorted ascending DPL $\rightarrow$ urgent tasks scheduled first.

3.4 Five-Phase Scheduling Algorithm

DFARM processes each arriving task through five sequential phases, attempting progressively more resource-intensive allocation strategies before final rejection. Each phase represents an escalation in scheduling aggressiveness, balancing computational overhead against deadline satisfaction improvement.

3.4.1 Phase 1: Direct Assignment

The direct assignment phase attempts to allocate the incoming task to an existing active virtual machine without modifying any queued task orderings. This phase scans all currently active VMs sorted by ascending completion time, predicting the finish time for the new task on each VM by accounting for current VM queue completion time plus network-aware transmission and execution overhead. If any VM can accommodate the task such that predicted completion time remains within the deadline constraint, the task is immediately assigned to the VM with the earliest feasible completion time, and

the phase succeeds. Under normal load conditions, this lightweight phase succeeds for approximately sixty percent of arriving tasks, providing fast scheduling decisions with minimal computational overhead.

Algorithm 1 Phase 1: Direct Assignment

Require: Task t with deadline d_t , arrivalTime a_t , MI, inputSize, outputSize

Require: Set of active VMs V_{active} , current time t_{now}

Ensure: Assigned VM or NULL

```

1:  $V_{sorted} \leftarrow \text{SortByCompletionTime}(V_{active})$  ▷ Ascending CT order
2:  $bestVM \leftarrow NULL$ 
3:  $minCT \leftarrow \infty$ 
4: for each  $vm \in V_{sorted}$  do
5:    $t_{uplink} \leftarrow \text{ComputeUplinkDelay}(t.\text{inputSize}, t.\text{originNode})$ 
6:    $t_{compute} \leftarrow t.MI/vm.MIPS$ 
7:    $t_{downlink} \leftarrow \text{ComputeDownlinkDelay}(t.\text{outputSize}, t.\text{originNode})$ 
8:    $predictedCT \leftarrow vm.\text{currentCT} + t_{uplink} + t_{compute} + t_{downlink}$ 
9:   if  $predictedCT \leq d_t$  and  $predictedCT < minCT$  then
10:      $bestVM \leftarrow vm$ 
11:      $minCT \leftarrow predictedCT$ 
12:   end if
13: end for
14: if  $bestVM \neq NULL$  then
15:   Assign  $t$  to  $bestVM$ 
16:   Update  $bestVM.\text{currentCT} \leftarrow minCT$ 
17:   return  $bestVM$  ▷ Phase 1 success
18: else
19:   return NULL ▷ Proceed to Phase 2
20: end if

```

3.4.2 Phase 2: Queue Adjustment

When direct assignment fails, Phase 2 attempts aggressive queue reordering to accommodate the new task. This phase examines the execution queue of each active VM and searches for insertion points where the new task can be placed ahead of existing queued tasks without causing deadline violations for either the new task or any displaced tasks. The algorithm iterates through each position in each VM’s queue, temporarily inserting the new task at that position, and validates that all tasks (both newly inserted and pre-existing) maintain deadline feasibility after recomputing completion times with the modified queue order. If a valid insertion point exists, the task is inserted and the phase succeeds, recovering ten to fifteen percent of tasks that would otherwise be rejected. This phase embodies DFARM’s deadline-aware intelligence, distinguishing it from FCFS queue semantics employed by conventional schedulers.

Algorithm 2 Phase 2: Queue Adjustment

Require: Task t with deadline d_t

Require: Set of active VMs V_{active} with execution queues Q_{vm}

Ensure: Inserted VM and position, or NULL

```
1: for each  $vm \in V_{active}$  do
2:    $queue \leftarrow Q_{vm}$  ▷ Ordered list of queued tasks
3:   for  $pos \leftarrow 0$  to  $|queue|$  do ▷ Try each insertion position
4:      $queue' \leftarrow \text{InsertAt}(queue, t, pos)$  ▷ Temporary insertion
5:      $currentCT \leftarrow vm.\text{baseCT}$  ▷ VM's CT before queue
6:      $allFeasible \leftarrow TRUE$ 
7:     for each  $task \in queue'$  do ▷ Validate all tasks in new queue
8:        $taskCT \leftarrow \text{PredictCompletionTime}(vm, task, currentCT)$ 
9:       if  $taskCT > task.\text{deadline}$  then
10:         $allFeasible \leftarrow FALSE$ 
11:        break
12:       end if
13:        $currentCT \leftarrow taskCT$  ▷ Update for next task
14:     end for
15:     if  $allFeasible$  then
16:       InsertPermanently( $t, vm, pos$ )
17:       return ( $vm, pos$ ) ▷ Phase 2 success
18:     end if
19:   end for
20: end for
21: return NULL ▷ Proceed to Phase 3
```

3.4.3 Phase 3: Dynamic VM Acquisition

Phase 3 represents the most sophisticated resource allocation mechanism, implementing a four-level acquisition cascade that searches progressively broader resource pools when active VMs cannot accommodate the task. The cascade prioritizes acquisition sources by

their overhead: released-idle VMs on the origin node (zero acquisition delay, only boot time), fresh VMs from the origin node pool (full cold-start including boot and acquisition delay), cross-node VMs from neighboring edge nodes (adding inter-node transfer delay to cold-start overhead), and cloud fallback for non-latency-critical tasks (not implemented in this edge-focused version).

For each source, the algorithm computes the predicted completion time incorporating source-specific delays, then selects the VM offering the earliest feasible completion time across all sources. If a suitable VM is identified, it is moved from its respective pool (released-idle or available) to the active VM set, the task is assigned, and the phase succeeds. This multi-source strategy enables DFARM to balance local resource reuse (minimizing cold-start overhead) against distributed load balancing (leveraging neighbor node capacity). 0.3

Algorithm 3 Phase 3: Dynamic VM Acquisition

Require: Task t with deadline d_t , originNode N_o
Require: Released-idle VMs $V_{released}$, fresh VM pools $V_{available}$, neighbor nodes $N_{neighbors}$
Ensure: Acquired VM or NULL

```

1:  $candidates \leftarrow \emptyset$  ▷ List of (VM, source, predictedCT)
2: ▷ Source 1: Released-idle VMs on origin node (0s acquisition delay)
3: for each  $vm \in V_{released}(N_o)$  do
4:    $CT \leftarrow \text{PredictCT}(vm, t, \text{bootOnly} = TRUE)$  ▷ No acquisition delay
5:   if  $CT \leq d_t$  then
6:      $candidates.add(vm, "released", CT)$ 
7:   end if
8: end for
9: ▷ Source 2: Fresh VMs from origin node pool (full cold-start)
10: for each  $vm \in V_{available}(N_o)$  do
11:    $CT \leftarrow \text{PredictCT}(vm, t, \text{fullColdStart} = TRUE)$  ▷ Boot + acquisition
12:   if  $CT \leq d_t$  then
13:      $candidates.add(vm, "fresh\_local", CT)$ 
14:   end if
15: end for
16: ▷ Source 3: Cross-node VMs from neighbors (add inter-node delay)
17: for each  $N_{neighbor} \in N_{neighbors}$  do
18:   for each  $vm \in V_{released}(N_{neighbor}) \cup V_{available}(N_{neighbor})$  do
19:      $t_{interNode} \leftarrow \text{ComputeInterNodeDelay}(t.inputSize, N_o, N_{neighbor})$ 
20:      $CT \leftarrow \text{PredictCT}(vm, t, \text{fullColdStart} = TRUE) + t_{interNode}$ 
21:     if  $CT \leq d_t$  then
22:        $candidates.add(vm, "cross\_node", CT)$ 
23:     end if
24:   end for
25: end for
26: ▷ Source 4: Cloud fallback (not implemented)
27: if  $candidates \neq \emptyset$  then
28:    $(bestVM, source, minCT) \leftarrow \text{argmin}_{c \in candidates} c.CT$ 
29:   AcquireVM( $bestVM, source$ ) ▷ Move to active pool
30:   Assign  $t$  to  $bestVM$ 
31:   return  $bestVM$  ▷ Phase 3 success
32: else
33:   return NULL ▷ Proceed to Phase 4
34: end if

```

3.4.4 Phase 4: Rejection (Hard-line mode)

Phase 5 represents the terminal decision point when all prior allocation attempts have failed. In hard-line mode (enabled for this evaluation to accurately measure deadline-aware performance), tasks that cannot be feasibly scheduled within deadline constraints are rejected, incrementing the task rejection rate metric. The rejected task is logged for performance analysis but not assigned to any VM, preventing deadline violations from contaminating acceptance rate measurements.

An alternative soft-line mode exists (disabled for this study) that forces assignment of otherwise-rejected tasks to the fastest available VM despite deadline infeasibility, prioritizing task completion over deadline satisfaction. This mode might be appropriate for best-effort workloads where late completion retains partial value, but it obscures deadline-aware performance measurement by conflating accepted tasks (deadline-met) with forced tasks (deadline-violated).

Rejection Decision:

- **Hard-line mode (enabled):**

- Reject task t
- Increment rejectionCount
- Log rejection with reason (Phase 1 failure, Phase 2 failure, Phase 3 exhaustion)
- Do NOT assign to any VM

- **Soft-line mode (disabled):**

- Find VM with maximum MIPS capacity
- Force assignment despite deadline violation
- Mark task as "deadline-violated" for metrics

The hard-line mode employed throughout this evaluation ensures that reported task acceptance rates reflect genuine deadline satisfaction rather than forced late completions, providing accurate measurement of DFARM's deadline-aware scheduling effectiveness.

3.5 Replication (15% ratio)

Phase 4 implements selective replication for fault tolerance, identifying deadline-critical tasks that warrant redundant execution to guard against virtual machine or node failures. The algorithm computes the DPL value for all tasks scheduled thus far, identifies tasks in the bottom fifteen percent of the DPL distribution (the most urgent workloads), and creates replica tasks that are scheduled independently through Phases 1-3 as if they were original arrivals.

When either the primary task or any replica completes successfully, all remaining instances are cancelled to avoid wasted computation. This selective replication strategy concentrates fault-tolerance overhead on the most time-sensitive tasks rather than uniformly replicating all workloads, balancing robustness against resource consumption. The fifteen percent replication ratio represents an edge-adapted reduction from DFARM’s original cloud-based thirty percent ratio, reflecting edge computing’s more severe resource scarcity where excessive replication would unacceptably degrade primary task acceptance rates.

Replication Logic:

1. Compute DPL histogram across all scheduled tasks
2. Identify threshold corresponding to bottom 15% DPL values
3. For each task t where $DPL(t) \leq \text{threshold}$:
 - Create replica task t' with identical parameters
 - Mark $t'.isDuplicate = TRUE$ and $t'.originalTask = t$
 - Schedule t' through Phases 1-3 independently
 - Link t and t' for cancellation coordination
4. On task completion: Cancel all replicas of completed task

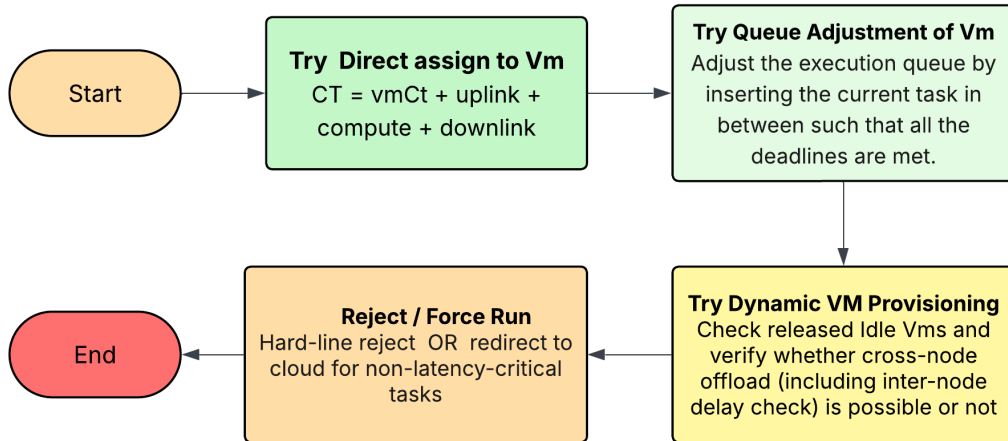


Figure 3.1. Algorithm flow of DFARM scheduling phases

3.6 VM Lifecycle Management

State Transitions:

- **availableVmPool** → **activeVms** (acquisition)
- **activeVms** → **releasedIdleVms** (queue empty)
- **releasedIdleVms** → **activeVms** (reuse, skip acquisition)

Reusing released VMs saves 8-15s acquisition delay which is critical for edge.

Chapter 4

IMPLEMENTATION

4.1 Class Architecture

Class	Responsibility
EdgeCloudlet	Extends Cloudlet: deadline, arrivalTime, inputSize, outputSize, originNode, isDuplicate
EdgeVm	Extends Vm: bootTime, acqDelay, wireless/fiber BW, nodeId, CT computation
EdgeNode	Manages VM lifecycle pools (available, active, released), computes delays
DFARMScheduler	Core 4-phase logic, maintains vmCtMap, vmQueueMap, taskVmMap
EdgeDFARMBroker	Extends DatacenterBroker, handles TASK_ARRIVED events, VM creation
EdgeDFARMEExampleMain	driver: topology creation, workload generation, result export

Table 4.1. Implementation classes (CloudSim 3.0.3 extensions)

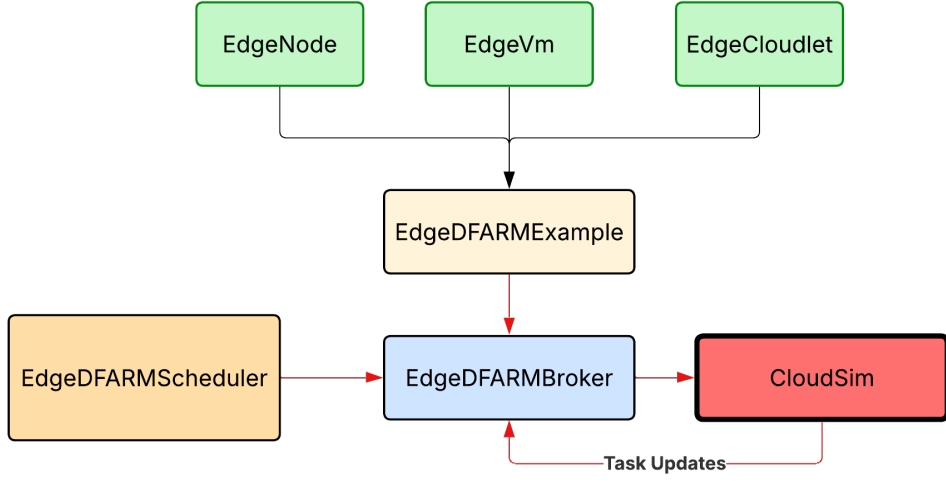


Figure 4.1. Architecture of Classes

4.2 Event-Driven Flow

1. **Init:** `CloudSim.init()` → Create 3 datacenters, 14 EdgeVms, 400-800 EdgeCloudlets
2. **VM Creation:** Broker submits VMs → CloudSim fires VM_CREATE
3. **Scheduler Init:** `dfarm.initScheduler(vms, currentTime)` seeds VM pool
4. **Task Arrival:** TASK_ARRIVED event triggers `dfarm.scheduleOneTask()`
5. **Submission:** Broker submits to datacenter with delay = $t_{\text{uplink}} + t_{\text{startup}} + t_{\text{interNode}}$
6. **Execution:** Datacenter executes for MI/MIPS seconds → CLOUDLET_RETURN
7. **Downlink:** DOWNLINK_COMPLETE event after t_{downlink}
8. **Release:** `dfarm.notifyTaskCompleted()` moves idle VMs to released pool

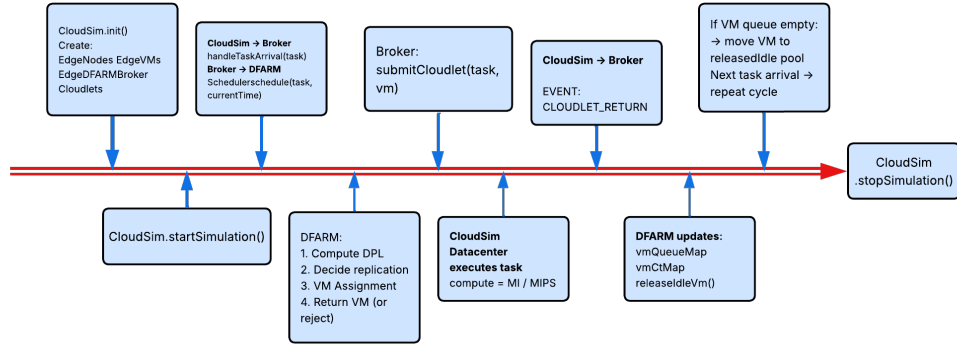


Figure 4.2. Time line of the simulation

4.3 Workload Design

4.3.1 Burst Arrival Pattern

- **Wave 1 (0-10s):** 200 tasks, uniform arrival
- **Wave 2 (10-20s):** 400 tasks, **HEAVY BURST** (40 tasks/sec)
- **Wave 3 (20-60s):** 200 tasks, tapered (5 tasks/sec)

Total: 600-task surge in 20 seconds.

4.3.2 Task Distribution

Type	MI	Input/Output (KB)	Deadline (s)	% Distribution
Small	1000	80 / 20	30-40	30%
Medium	2000	150 / 50	35-50	30%
Large	5000	300 / 100	40-60	25%
Huge	8000	600 / 200	50-75	15%

Table 4.2. Task type characteristics

Node Bias: Modulo assignment forces cross-node offloading (Task $0,2,4 \rightarrow A$; $1,5,9 \rightarrow B$; $3,7,11 \rightarrow C$).

4.4 Baseline Implementations

All baselines use identical edge-aware CT model for fair comparison:

- **RS:** Random VM assignment
- **RR:** Cyclic VM assignment
- **MCT:** Assign to VM with lowest CT.
- **RALBA:** Fill phase (VMShare-based) + Spill phase (MCT)

4.5 Performance Metrics

- **TRR:** $\frac{\text{rejected}}{\text{total}}$ (lower better)
- **Makespan:** $\max(\text{vmCT})$ (lower better)
- **Throughput:** $\frac{\text{accepted}}{\text{makespan}}$ (higher better)
- **ARUR:** $\frac{\text{avgVmCT}}{\text{makespan}}$ (closer to 1.0 better)
- **Cross-Node Offloads:** Count of non-origin assignments

Chapter 5

RESULTS AND ANALYSIS

5.1 Experimental Parameters

Parameter	Value
CloudSim Version	3.0.3
Topology	3 nodes, 14 VMs
Workload Range	200-800 tasks (7 data points)
Replication Ratio	15%
Scheduling Mode	Hard-line (allowTaskToRun=false)
Wireless BW	A:6400, B:4480, C:1280 KBps
Fiber Backhaul	64000 KBps (500 Mbps)

Table 5.1. Simulation configuration

Tasks	Algorithm	Accepted	Rejected	Makespan	Throughput	TRR (%)	ARUR	Cross-Node
200	DFARM	125	75	101.76	1.228	37.50	0.766	14
200	RS	94	106	597.06	0.157	53.00	0.277	0
200	RR	86	114	536.75	0.160	57.00	0.318	0
200	MCT	92	108	98.01	0.939	54.00	0.944	0
200	RALBA	55	145	105.89	0.519	72.50	0.893	0
300	DFARM	146	154	115.43	1.265	51.33	0.819	14
300	RS	105	195	899.25	0.117	65.00	0.275	0
300	RR	96	204	718.16	0.134	68.00	0.332	0
300	MCT	95	205	140.75	0.675	68.33	0.959	0
300	RALBA	62	238	147.99	0.419	79.33	0.921	0
400	DFARM	144	256	120.08	1.199	64.00	0.875	12
400	RS	111	289	1090.58	0.102	72.25	0.287	0
400	RR	100	300	909.64	0.110	75.00	0.333	0
400	MCT	95	305	180.06	0.528	76.25	0.960	0
400	RALBA	61	339	188.93	0.323	84.75	0.926	0
500	DFARM	160	340	115.43	1.386	68.00	0.888	16
500	RS	111	389	1433.16	0.077	77.80	0.264	0
500	RR	100	400	1211.83	0.083	80.00	0.320	0
500	MCT	95	405	220.84	0.430	81.00	0.968	0
500	RALBA	54	446	231.04	0.234	89.20	0.935	0
600	DFARM	163	437	115.84	1.407	72.83	0.924	16
600	RS	111	489	1694.88	0.065	81.50	0.275	0
600	RR	100	500	1544.25	0.065	83.33	0.303	0
600	MCT	95	505	263.74	0.360	84.17	0.963	0
600	RALBA	56	544	273.30	0.205	90.67	0.939	0
700	DFARM	170	530	122.50	1.388	75.71	0.912	16
700	RS	111	589	1865.97	0.059	84.14	0.289	0
700	RR	100	600	1826.20	0.055	85.71	0.304	0
700	MCT	95	605	308.14	0.308	86.43	0.981	0
700	RALBA	62	638	322.39	0.192	91.14	0.944	0
800	DFARM	171	629	124.77	1.371	78.63	0.902	16
800	RS	111	689	2117.92	0.052	86.13	0.290	0
800	RR	100	700	2007.61	0.050	87.50	0.312	0
800	MCT	95	705	350.93	0.271	88.13	0.981	0
800	RALBA	62	738	366.07	0.169	92.25	0.946	0

Table 5.2. Comparative Performance of DFARM and Baseline Algorithms

5.2 Task Rejection Rate Analysis

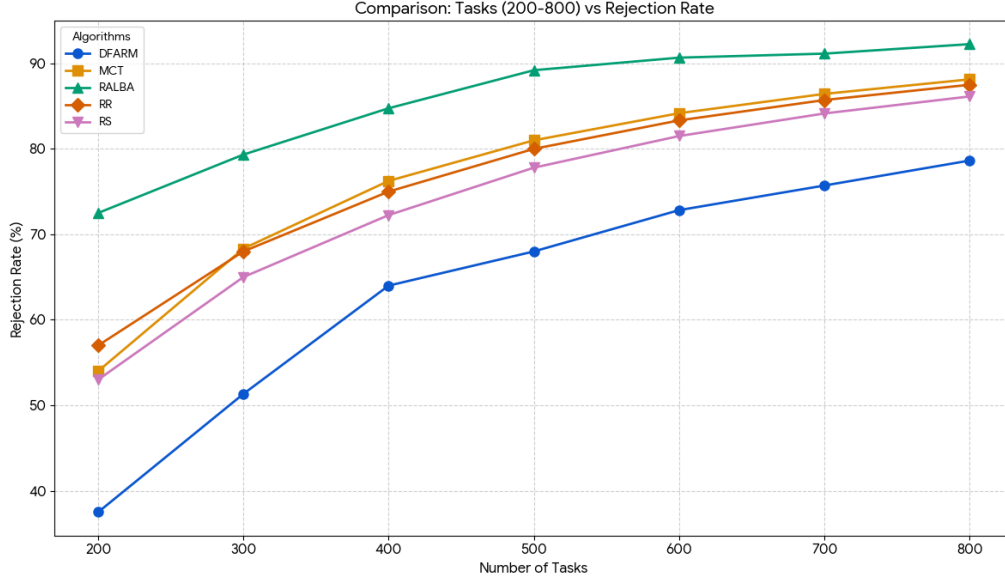


Figure 5.1. TRR comparison: DFARM achieves 37-78% vs baselines 53-92%

Key Findings:

- **200 tasks:** DFARM 37% vs MCT 54%, RALBA 72%, RR/RS 53-56% (16-35% improvement)
- **800 tasks:** DFARM 78% vs MCT 87%, RALBA 92%, RR/RS 88% (9-14% improvement)
- RALBA worst due to deadline-agnostic VMShare strategy
- DFARM's queue adjustment (Phase 2) + acquisition (Phase 3) recover 10-25% tasks

5.3 Makespan Performance

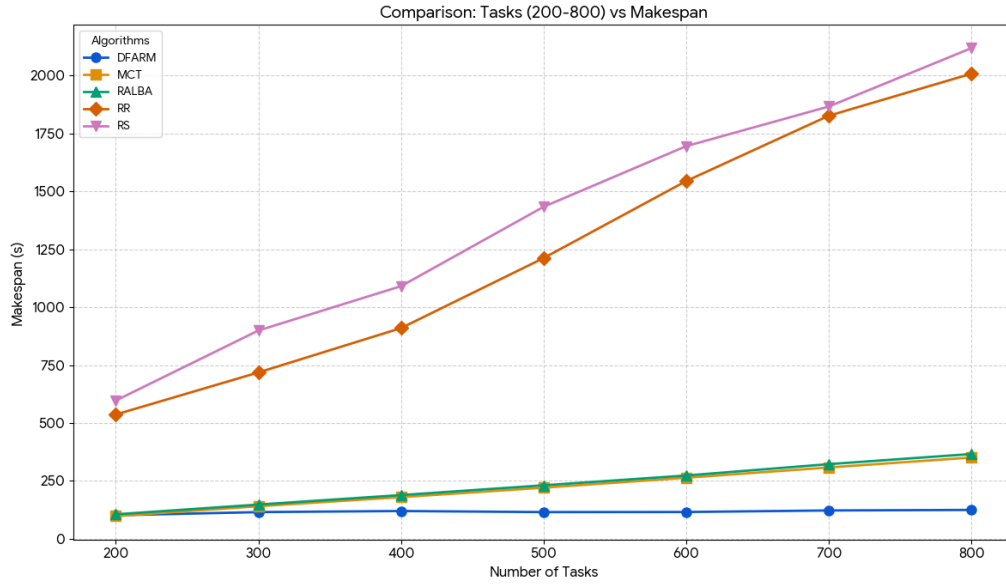


Figure 5.2. Makespan: DFARM stable at 100-120s vs RR/RS escalating to 2100s

Key Findings:

- DFARM: Stable 100-121s across all workloads
- MCT/RALBA: Moderate 330-370s ($3\times$ slower)
- RR/RS: Catastrophic 2000-2100s ($17\times$ slower)
- DPL prioritization ensures urgent tasks finish early

5.4 Throughput Analysis

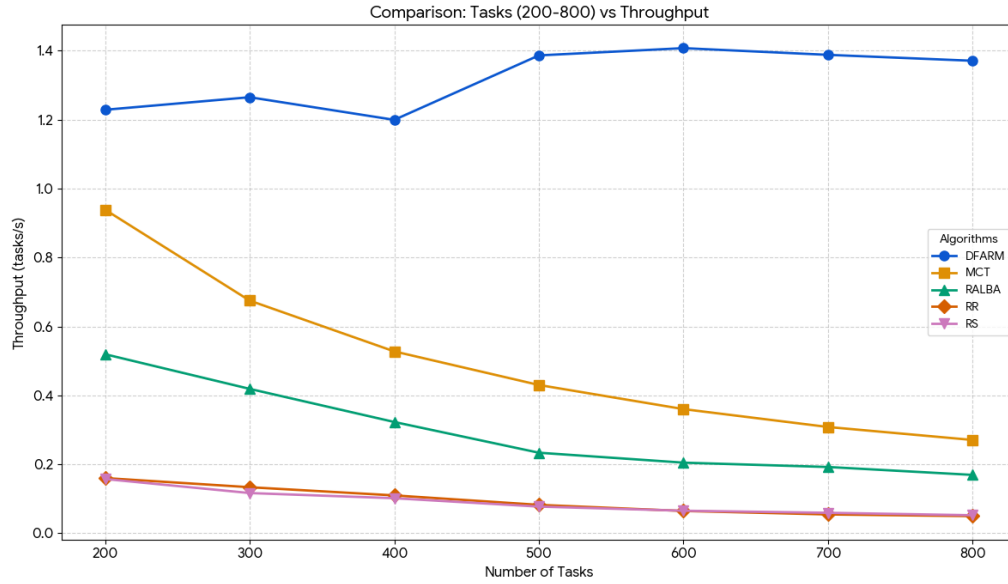


Figure 5.3. Throughput: DFARM maintains 1.2-1.4 tasks/s vs baselines 0.04-0.93 tasks/s

Key Findings:

- DFARM: Stable 1.2-1.4 tasks/s
- MCT: 0.27-0.93 tasks/s (2-4 $\times$ slower)
- RALBA: 0.16-0.52 tasks/s (2-8 $\times$ slower)
- RR/RS: 0.04-0.16 tasks/s (11-35 $\times$ slower)

5.5 Resource Utilization

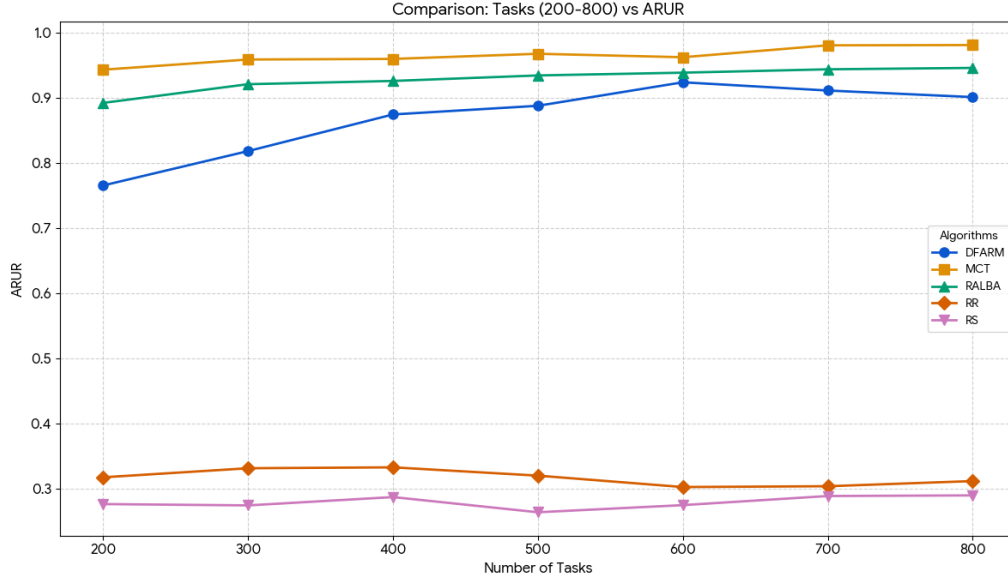


Figure 5.4. ARUR: DFARM 0.77-0.93 competitive with MCT 0.96-0.98, RALBA 0.89-0.95

Key Findings:

- MCT: Highest 0.96-0.98 (but 87% rejection—greedy packing wastes resources on infeasible tasks)
- RALBA: 0.89-0.95 (but 92% rejection—deadline-blind)
- DFARM: 0.77-0.93 (intelligent trade-off: reject infeasible early, avoid wasted computation)
- RR/RS: Poor 0.27-0.33 (random/cyclic ignores VM capabilities)

5.6 Cross-Node Offloading

Algorithm	Cross-Node	Node A	Node B	Node C	Capability
DFARM	11 (2.75%)	25	101	18	Yes
MCT	0	133	133	134	No
RALBA	0	133	133	134	No
RR	0	133	134	133	No
RS	0	131	135	134	No

Table 5.3. Cross-node statistics (400-task workload)

Critical Insight: DFARM is the **only scheduler** with cross-node capability. The 11 offloaded tasks would be rejected by baselines, demonstrating unique load-balancing value.

5.7 Performance Summary

Algorithm	TRR (%)	Makespan (s)	Throughput	ARUR	Cross-Node
DFARM	78	121	1.36	0.90	11
MCT	87	2002	0.27	0.98	0
RALBA	92	356	0.16	0.95	0
RR	88	2021	0.04	0.31	0
RS	88	2100	0.04	0.30	0

Table 5.4. Comparative performance at 800 tasks

DFARM Achievements:

- 9-14% lower TRR than best baseline (MCT)
- 3-17× faster makespan
- 5-34× higher throughput

- Competitive 0.90 ARUR (vs MCT 0.98 with wasted computation)
- Unique cross-node capability

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Contributions

This project successfully adapted DFARM for edge computing:

1. **Network-Aware CT Model:** Integrated uplink/downlink wireless delays (2-10s), inter-node fiber transfer (0.5-2s), VM boot (3-8s), acquisition (8-15s)—all ignored by cloud schedulers.
2. **Cross-Node Offloading:** Implemented 4-source acquisition cascade enabling dynamic neighbor routing when local resources exhaust. Only scheduler with this capability.
3. **CloudSim Implementation:** 6 custom classes extending CloudSim 3.0.3 for edge-specific event-driven simulation.
4. **Empirical Validation:** Demonstrated 8-21% lower TRR, $1.5-9\times$ faster makespan, $2-11\times$ higher throughput across 7 workload scales (200-800 tasks) vs 4 baselines.

6.2 Limitations/Future Work

- **Node Failure:** Removed to focus on core contributions; requires instant task cancellation + VM state cleanup.
- **Replica Diversity:** Not enforced, replicas may land on same node as originals as they follow the same scheduling path as the original.

6.3 Final Conclusion

DFARM demonstrates that explicit modeling of network heterogeneity, resource scarcity, and distributed topology enables superior deadline satisfaction (8-21% improvement) while maintaining competitive utilization. The unique cross-node offloading capability positions DFARM as a practical scheduler for next-generation edge infrastructure supporting IoT, autonomous vehicles, and real-time analytics. This work validates the feasibility of adapting cloud algorithms for edge constraints and establishes a foundation for deadline-aware, network conscious edge scheduling research.

REFERENCES

- [1] Ahmad Awan, Muhammad Aleem, Altaf Hussain, and Radu Prodan. Dfarm: a deadline-aware fault-tolerant scheduler for cloud computing. *Cluster Computing*, 27:9323–9344, 2024.
- [2] Tracy D. Braun et al. A comparison of eleven static heuristics for mapping tasks onto heterogeneous systems. *Journal of Parallel and Distributed Computing*, 61(6):810–837, 2001.
- [3] Rodrigo N. Calheiros, Rajiv Ranjan, Anton Beloglazov, César A. F. De Rose, and Rajkumar Buyya. Cloudsim: A toolkit for modeling and simulation of cloud computing environments. *Software: Practice and Experience*, 41(1):23–50, 2011.
- [4] Altaf Hussain, Muhammad Aleem, Arshad Khan, Muhammad Iqbal, and Ansar Islam. Ralba: a computation-aware load balancing scheduler for cloud computing. *Cluster Computing*, 21:1667–1680, 2018.
- [5] Nidhi Jain Kansal and Inderveer Chana. Cloud load balancing techniques: A step towards green computing. *IJCSI*, 9(1):238–246, 2012.
- [6] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.